

Table of content

Preamble.....	2
Not chip.....	3
And chip.....	3
Or chip.....	3
Xor chip.....	4
Mux chip.....	4
DMux chip.....	4
Not16 chip.....	5
And16 chip.....	5
Or16 chip.....	5
Mux16 chip.....	5
Or8Way chip.....	5
Mux4Way16 chip.....	5
Mux8Way16 chip.....	6
DMux4Way chip.....	6
DMux8Way chip.....	6

Preamble

The present document gives some informations upon the HDL code available in the [01](#) Bitbucket directory. The code may not be optimal nor elegant but it passes all the tests supplied in the Nand2Tetris project files.

Not chip

Truth tables :

a	not a
0	1
1	0

a	a	a nand a
0	0	1
1	1	0

So : $\text{not}(a) = a \text{ nand } a$

And in HDL : `Nand(a=in, b=in, out=out);`

And chip

Since $a \text{ nand } b = \text{not}(a \text{ and } b)$, $a \text{ and } b = \text{not}(a \text{ nand } b)$. So $a \text{ and } b = (a \text{ nand } b) \text{ nand } (a \text{ nand } b)$ based on the not chip.

In HDL, the result of $a \text{ nand } b$ is stored into $n1$ and then we compute $n1 \text{ nand } n1$:

```
Nand(a=a, b=b, out=n1);
Nand(a=n1, b=n1, out=out);
```

or chip

Truth tables :

a	b	a or b	not(a)	not(b)	not(a) and not(b)	not(not(a) and not(b))
0	0	0	1	1	1	0
0	1	1	1	0	0	1
1	0	1	0	1	0	1
1	1	1	0	0	0	1

So : $a \text{ or } b = \text{not}(\text{not}(a) \text{ and } \text{not}(b)) = \text{not}(a) \text{ nand } \text{not}(b)$

In HDL :

```
Nand(a=a, b=a, out=nota);
Nand(a=b, b=b, out=notb);
Nand(a=nota, b=notb, out=out);
```

Xor chip

The `xor.hd1` file contains 2 versions. The first one uses only `Nand` gates. The second one (commented in the code) uses `Not`, `And` and `Or` gates. The translation can be seen through the following truth table :

a	b	not(a)	not(b)	not(a) and b	a and not(b)	(not(a) and b) or (a and not(b))
0	0	1	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

And also through the fact that $a \text{ xor } b$ can be seen as $(a \text{ or } b) \text{ and } \text{not}(a \text{ and } b)$. Which can be expressed as $(a \text{ and } \text{not}(b)) \text{ or } (b \text{ and } \text{not}(a))$. I don't know if there is a simpler way or a more elegant way to express the chip.

Mux chip

From the definition of the chip, $\text{mux}(a, b, \text{sel}) = (a \text{ and } \text{not}(\text{sel})) \text{ or } (b \text{ and } \text{sel})$ which can be translated as $(a \text{ nand } (\text{sel} \text{ nand } \text{sel})) \text{ nand } (b \text{ nand } \text{sel})$. And can be verified through the following truth table :

a	b	sel	sel nand sel	a nand (sel nand sel)	b nand sel	(a nand (sel nand sel)) nand (b nand sel)
0	0	0	1	1	1	0
0	1	0	1	1	1	0
1	0	0	1	0	1	1
1	1	0	1	0	1	1
0	0	1	0	1	1	0
0	1	1	0	1	0	1
1	0	1	0	1	1	0
1	1	1	0	1	0	1

DMux chip

From the definition of the chip, $\text{dmux}(\text{in}, \text{sel}) = (\text{in}, 0)$ if $\text{sel} = 0$ or $(0, \text{in})$ if $\text{sel} = 1$. Which has been translated like this in `Dmux.hd1` :

```
// a = in And Not(sel)
// b = in And sel
Not(in=sel, out=nsel);
And(a=in, b=nsel, out=a);
And(a=in, b=sel, out=b);
```

Not16 chip

From the definition of the chip, we have $in[16]$ as input, $out[16]$ as output and $out[i] = \text{Not}(in[i])$ for i in $[0, 15]$. Which can give the following HDL code :

```
Not(in=in[0], out=out[0]);
Not(in=in[1], out=out[1]);
(...)
Not(in=in[15], out=out[15]);
```

The following command line for loops can generate the previous code automatically :

- Windows : `for /L %N in (0, 1, 15) do @echo Not(in=in[%N], out=out[%N]);`
- Bash : `for N in {0..15}; do echo "Not(in=in[${N}], out=out[${N}]);" ; done`

And16 chip

Straightforward implementation from the previous chip.

Or16 chip

Likewise.

Mux16 chip

Likewise.

Or8Way chip

Since $out = \text{Or}(in[0], in[1], \dots, in[7])$, we first compute $or1 = \text{Or}(in[0], in[1])$ and then $or2 = \text{Or}(or1, in[2])$ and so on until $out = \text{Or}(or6, in[7])$.

Mux4Way16 chip

For this chip, we begin by computing four 1-bit selectors : sel_a , sel_b , sel_c and sel_d .

If the chip has to select a as output, we will have $sel_a = 0$ (or 1 else).

If the chip has to select b as output, we will have $sel_b = 0$ (or 1 else). And so on for sel_c and sel_d .

One can check in the HDL code that one, and only one, selector will be set to 0 after the last or step has been run.

Once the selectors computed, we step through the following Mux16 chips :

- `Mux16(a=a, b=b, sel=sel_a, out=outa)` insures that out_a will be set to a if $sel_a = 0$ or b else.
- `Mux16(a=b, b=outa, sel=sel_b, out=outb)` insures that out_b will be set to b if $sel_b = 0$ or out_a else.
- `Mux16(a=c, b=outb, sel=sel_c, out=outc)` insures that out_c will be set to c if $sel_c = 0$ or out_b else.
- `Mux16(a=d, b=outc, sel=sel_d, out=out)` insures that out will be set to d if $sel_d = 0$ or out_c else.

Since only one of the four selectors is 0, one can also check that the right input will land into out .

Mux8Way16 chip

This chip is a « simple » generalization of the previous chip. The number of lines of code is larger (nearly x 3) but the idea is the same.

DMux4Way chip

Here, we start again by computing four 1-bit selectors : `sela`, `selb`, `selc` and `seld`. Each one will tell if the corresponding output pin (a, b, c or d) will be set to `in` or to `0`. This computation can be summarized by the following table :

<code>sel[1]</code>	<code>sel[0]</code>	<code>notsel1</code>	<code>notsel0</code>	<code>sela</code>	<code>selb</code>	<code>selc</code>	<code>seld</code>
0	0	1	1	1	0	0	0
0	1	1	0	0	1	0	0
1	0	0	1	0	0	1	0
1	1	0	0	0	0	0	1

Same remark as for the Mux4Way16 chip : one and only one selector will be set to 1 after the fourth selector has been initialized. After that, we simply *and* the input with each selector to set the matching output pin :

```
And(a=in, b=sela, out=a);  
And(a=in, b=selb, out=b);  
And(a=in, b=selc, out=c);  
And(a=in, b=seld, out=d);
```

DMux8Way chip

Same implementation principle as the previous chip but with more lines of code.